

CRAFT: Supplementary Experimental Analyses

This supplement examines robustness, systems cost, and design sensitivity. Appendix A tests whether duplicate instances within the evaluation benchmark change the conclusions of the Qwen3-8B training experiments. Appendix B characterizes the computational profile of the evaluation used for Qwen3-8B and all of its training variants. Appendix C isolates the effect of negative-trace sampling under the Full reward. Appendix D reports three training runs with three inference runs each. Appendix E archives a rerun of the main paper’s tool comparison with medium reasoning effort. Appendix F examines filtering and cross-response pooling on a matched Qwen3-8B run. All verification rates are percentages.

A Robustness to Benchmark Deduplication

A.1 Question and protocol

The full benchmark contains 832 programs. A separate audit identifies 124 evaluation instances as duplicates of other instances within this benchmark; removing those redundant instances leaves 708 programs. We evaluate sensitivity to these within-benchmark duplicates while keeping the model checkpoints, saved response order, sampling budgets $k \in \{1, 4, 8, 16, 32\}$, composition procedure, and verifier unchanged.

We report two complementary comparisons. The *training-stage comparison* follows representative Base, RL-from-Base, and SFT+RL checkpoints. The *matched reward ablation* separately compares Binary, Whole-rollout, Clause-decomposed, and Full reward runs within a common initialization. The two comparisons use separate checkpoint groups.

A.2 Training-stage comparison

Tables 1 and 2 pair the full and internally deduplicated evaluations. Parenthesized entries on the 708-program rows are percentage-point changes from the corresponding 832-program result.

Setting	N	@1	@4	@8	@16	@32
Base	832	43.75	55.17	59.01	61.18	62.50
	708	42.09 (-1.66)	53.81 (-1.36)	57.91 (-1.10)	60.17 (-1.01)	61.44 (-1.06)
RL from Base	832	51.80	60.46	64.18	68.87	70.67
	708	48.87 (-2.93)	57.20 (-3.26)	61.16 (-3.02)	65.68 (-3.19)	67.51 (-3.16)
SFT+RL	832	66.23	72.48	74.40	75.12	75.60
	708	62.99 (-3.24)	69.07 (-3.41)	71.05 (-3.35)	71.89 (-3.23)	72.32 (-3.28)

Table 1: Qwen3-8B training-stage comparison under compose@ k .

Setting	N	@1	@4	@8	@16	@32
Base	832	8.29	16.47	20.55	24.40	28.13
	708	7.91 (-0.38)	15.82 (-0.65)	19.77 (-0.78)	23.31 (-1.09)	26.84 (-1.29)
RL from Base	832	2.88	5.41	7.81	9.62	11.06
	708	2.54 (-0.34)	4.24 (-1.17)	6.64 (-1.17)	7.91 (-1.71)	9.18 (-1.88)
SFT+RL	832	32.93	46.88	51.44	53.37	55.89
	708	30.37 (-2.56)	43.64 (-3.24)	47.88 (-3.56)	49.72 (-3.65)	52.40 (-3.49)

Table 2: Qwen3-8B training-stage comparison under pass@ k .

Stage conclusions. Removing within-benchmark duplicates lowers the reported rates but preserves the shape of both curves. Across the two tables, the absolute changes range from 0.30 to 3.65 points. The larger shifts for SFT+RL indicate that the removed subset is comparatively favorable to that checkpoint, but SFT+RL remains the strongest stage at every budget under both composition and pass. Likewise, RL from Base continues to improve $\text{compose}@k$ over Base while reducing $\text{pass}@k$, preserving the distinction between standalone-response quality and cross-response clause utility.

A.3 Stability of the reward ablation

The central robustness question is whether internal benchmark deduplication changes the reward ranking. Tables 3 and 4 report the complete matched ablation. Each reward has adjacent 832- and 708-program rows. “Base (reference)” appears only in the Base-initialization panel; it is not an additional trained reward.

Reward	N	@1	@4	@8	@16	@32
<i>RL directly from Base</i>						
Base (reference)	832	43.75	55.17	59.01	61.18	62.50
	708	42.09	53.81	57.91	60.17	61.44
Binary	832	4.45	4.45	4.45	4.45	4.45
	708	4.52	4.52	4.52	4.52	4.52
Whole-rollout	832	20.79	21.88	22.24	22.84	23.32
	708	19.63	20.76	21.19	21.61	22.03
Clause-decomposed	832	48.80	57.69	60.70	63.70	66.35
	708	46.75	55.93	59.04	61.86	64.69
Full (ours)	832	51.80	60.46	64.18	68.87	70.67
	708	48.87	57.20	61.16	65.68	67.51
<i>RL after SFT</i>						
Binary	832	7.09	10.82	13.34	14.54	15.99
	708	6.78	10.03	12.57	13.84	15.54
Whole-rollout	832	48.80	56.37	60.22	63.10	66.59
	708	47.03	53.67	57.06	59.89	63.42
Clause-decomposed	832	67.91	75.84	76.80	77.64	78.25
	708	64.69	72.88	73.73	74.72	75.14
Full (ours)	832	68.51	75.72	77.52	78.25	78.37
	708	64.83	72.46	74.58	75.28	75.42

Table 3: $\text{Compose}@k$ reward ablation before and after benchmark deduplication.

Reward	<i>N</i>	@1	@4	@8	@16	@32
<i>RL directly from Base</i>						
Base (reference)	832	8.29	16.47	20.55	24.40	28.13
	708	7.91	15.82	19.77	23.31	26.84
Binary	832	4.45	4.45	4.45	4.45	4.45
	708	4.52	4.52	4.52	4.52	4.52
Whole-rollout	832	15.50	16.59	16.83	17.67	18.39
	708	15.54	16.67	16.95	17.66	18.08
Clause-decomposed	832	9.01	15.87	18.87	20.31	21.63
	708	8.19	14.69	17.66	19.21	20.62
Full (ours)	832	2.88	5.41	7.81	9.62	11.06
	708	2.54	4.24	6.64	7.91	9.18
<i>RL after SFT</i>						
Binary	832	8.77	11.30	13.22	14.30	15.50
	708	8.47	10.73	12.57	13.70	15.11
Whole-rollout	832	41.11	49.28	53.13	55.53	58.29
	708	38.56	46.05	49.58	51.84	54.66
Clause-decomposed	832	35.94	51.44	54.33	57.69	61.30
	708	32.06	47.32	50.56	53.39	56.78
Full (ours)	832	36.78	51.92	56.49	59.50	61.54
	708	32.91	47.88	52.40	55.23	57.34

Table 4: Pass@*k* reward ablation before and after benchmark deduplication.

The composition result is robust. Clause-level rewards dominate response-level rewards under compose@*k* on both evaluation sets and from both initializations. At compose@32 the strict ordering is unchanged:

Full > Clause-decomposed > Whole-rollout > Binary.

After SFT, Full and Clause-decomposed are close: Clause-decomposed is slightly higher at @4, whereas Full is higher at @1, @8, @16, and @32 on both sets. These small local crossings do not alter the endpoint ranking or the larger gap between clause-level and response-level credit.

Compose and pass expose different effects. Whole-rollout can perform substantially worse under composition than the clause-level objectives, especially from Base, while retaining stronger pass@*k*. This is consistent with the objectives: response-level credit favors complete standalone answers, whereas clause-level credit preserves useful partial content for later composition. The within-benchmark deduplication control retains this contrast.

B Cost and Systems Profile

B.1 Measurement scope

We profile the common evaluation configuration used for the Qwen3-8B base model and all of its training variants. Each checkpoint is evaluated on all 832 programs with eight rollouts per task. Totals report the cost of one complete checkpoint evaluation. The measurement boundary includes prompt processing, response generation, candidate extraction and pooling, Houdini filtering, and final Frama-C/WP verification. Token totals, verifier timings, and memory measurements are recorded quantities. Multi-GPU generation time and the 80-worker wall-clock decomposition are estimated from measured throughput and aggregate work for this serving configuration.

B.2 Generation workload

Component	Mean per task	Total over 832 tasks
Prompt tokens	1,113	925,635
Output tokens (8 rollouts)	11,057	9,199,244
Total tokens	12,169	10,124,879

Table 5: Token consumption for one Qwen3-8B evaluation with eight responses per task.

Each rollout produces 1,382 output tokens on average and yields approximately 19.6 parsed clauses. Pooling and deduplicating the first eight rollouts leaves 58.8 candidate invariants per task on average, with a maximum of 106. Thus, candidate deduplication removes substantial textual repetition before formal verification: eight responses yield roughly 157 parsed clauses in total, but only 58.8 distinct candidates enter the pooled set on average.

Configuration	Throughput or time
A800-80G throughput	5,833 output tokens/s/GPU
One-GPU generation	26.3 min
Four-GPU generation	6.6 min

Table 6: Measured vLLM output throughput and derived generation time.

The one-GPU time follows from the 9,199,244 generated tokens and measured output-token throughput. The four-GPU value assumes even task allocation at the same per-GPU throughput. Generation is therefore inexpensive relative to the subsequent proof workload once multiple GPUs are available.

B.3 Formal verification dominates compute

Subset	N	Mean	Median	P95	CPU time	Round budget hits
All	832	123.4 s	25.5 s	551.1 s	28.5 h	55 (6.6%)
Linear	316	112.7 s	24.6 s	530.7 s	9.9 h	—
NLA	50	292.0 s	252.6 s	866.8 s	4.1 h	—
Loopy	466	112.6 s	22.3 s	564.5 s	14.6 h	—

Table 7: Frama-C/Houdini verification cost overall and by benchmark subset.

Per-task verification time accumulates work across Houdini rounds, whereas the 300-second budget applies to each individual round. Total task time can therefore exceed 300 seconds. The distribution is strongly right-skewed: the overall median is 25.5 seconds, the P95 is 551.1 seconds, and 55 tasks have at least one round that reaches the budget. NLA is the most expensive subset per task. Although it contains only 50 programs, it consumes 4.1 CPU-hours and averages 292.0 seconds (4.9 minutes) per program. Loopy consumes the most aggregate CPU time because it is the largest subset, not because its per-task mean exceeds Linear.

B.4 Memory and parallel execution

Component	Memory or concurrency
GPU model weights	15.27 GiB
GPU KV cache	55.43 GiB (capacity: approximately 403K tokens)
Total per GPU	approximately 73.6/80 GiB
CPU per Frama-C worker	approximately 200–500 MB
Concurrent CPU workers	60–100

Table 8: Memory footprint and verifier concurrency.

The 80-GiB GPU is close to capacity, but most allocated memory is the KV cache rather than model weights. On the verifier side, 80 concurrent workers imply approximately 16–40 GB of worker memory from the observed per-process range, excluding the

operating system and orchestration overhead. The pipeline is therefore GPU-memory-bound during serving and CPU-parallelism-bound during formal verification.

B.5 End-to-end bottleneck

Stage	Wall-clock time	Share
LLM generation	6.6 min	23%
Frama-C verification	21.4 min	77%
Total	approximately 28 min	–

Table 9: End-to-end wall-clock decomposition with four GPUs and 80 CPU verification workers.

The systems result mirrors the CPU accounting: proof is the bottleneck. Verification occupies 77% of the estimated end-to-end wall-clock time. Additional generation throughput alone would therefore have limited impact; reducing proof obligations, avoiding repeated verifier work, or improving verifier load balancing targets the dominant cost.

C Negative-Trace Sampling Ablation

We next isolate negative-trace sampling while holding the Full reward fixed. Structure uses the target-hidden negative-trace construction defined in the main paper; Random replaces it with a budget-matched random construction. We compare both samplers under RL and SFT+RL. As in the main paper, RL denotes training directly from Base, whereas SFT+RL starts from the SFT model.

Training	Sampler	@1	@4	@8	@16	@32
SFT+RL	Structure	67.55	76.08	77.52	78.61	78.73
	Random	68.39	75.24	77.16	78.25	78.49
	Δ	-0.84	+0.84	+0.36	+0.36	+0.24
RL	Structure	49.28	57.93	62.26	64.90	66.83
	Random	50.48	59.86	62.50	64.78	66.11
	Δ	-1.20	-1.93	-0.24	+0.12	+0.72

Table 10: Qwen3-8B negative-sampling ablation with Full reward: $\text{compose}@k$. Differences are Structure minus Random in percentage points.

Training	Sampler	@1	@4	@8	@16	@32
SFT+RL	Structure	37.98	51.80	56.13	59.98	62.74
	Random	37.26	48.20	53.85	58.05	60.70
	Δ	+0.72	+3.60	+2.28	+1.93	+2.04
RL	Structure	8.17	13.10	14.90	17.43	19.35
	Random	5.65	11.18	13.22	15.87	18.03
	Δ	+2.52	+1.92	+1.68	+1.56	+1.32

Table 11: Qwen3-8B negative-sampling ablation with Full reward: $\text{pass}@k$. Differences are Structure minus Random in percentage points.

Tables 12 and 13 disaggregate the same comparison by program class. The “All” results are reported separately above to avoid repeating the aggregate in the stratified tables.

Composition is insensitive to negative-trace sampling. For $\text{compose}@k$, the aggregate Structure–Random difference lies between -1.93 and +0.84 points and changes sign with the training stage and response budget. The class-level results show the same mixed pattern: neither sampler dominates uniformly across Linear, NLA, and Loopy programs. Thus, Structure has no systematic effect on $\text{compose}@k$, despite the modest and consistent improvements in $\text{pass}@k$ reported below. This ablation also separates negative-trace sampling from the larger reward-design effects above: the qualitative advantage of clause-level credit under composition cannot be attributed to the sampling strategy.

Training	Sampler	@1	@4	@8	@16	@32
<i>Linear</i> ($n = 316$)						
SFT+RL	Structure	69.94	79.11	80.70	81.33	81.33
	Random	72.47	78.48	80.70	81.33	81.33
RL	Structure	53.16	62.97	66.77	70.25	71.20
	Random	52.53	62.34	65.19	66.77	68.99
<i>NLA</i> ($n = 50$)						
SFT+RL	Structure	12.00	14.00	14.00	14.00	14.00
	Random	12.00	12.00	12.00	12.00	12.00
RL	Structure	10.00	12.00	12.00	12.00	12.00
	Random	12.00	12.00	12.00	14.00	14.00
<i>Loopy</i> ($n = 466$)						
SFT+RL	Structure	71.89	80.69	82.19	83.69	83.91
	Random	71.67	79.83	81.76	83.26	83.69
RL	Structure	50.86	59.44	64.59	66.95	69.74
	Random	53.22	63.30	66.09	68.88	69.74

Table 12: Compose@ k negative-trace sampling ablation by program class.

Training	Sampler	@1	@4	@8	@16	@32
<i>Linear</i> ($n = 316$)						
SFT+RL	Structure	40.51	56.33	59.18	64.56	67.09
	Random	37.66	49.68	56.01	59.49	63.61
RL	Structure	10.13	13.29	15.19	18.35	20.25
	Random	8.23	13.29	15.19	17.41	19.62
<i>NLA</i> ($n = 50$)						
SFT+RL	Structure	6.00	6.00	8.00	8.00	8.00
	Random	6.00	8.00	8.00	8.00	8.00
RL	Structure	0.00	0.00	2.00	2.00	2.00
	Random	0.00	0.00	2.00	2.00	2.00
<i>Loopy</i> ($n = 466$)						
SFT+RL	Structure	39.70	53.65	59.23	62.45	65.67
	Random	40.34	51.50	57.30	62.45	64.38
RL	Structure	7.73	14.38	16.09	18.45	20.60
	Random	4.51	10.94	13.09	16.31	18.67

Table 13: Pass@ k negative-trace sampling ablation by program class.

Negative-trace sampling also affects standalone success. On the full benchmark, Structure improves $\text{pass}@k$ over Random at every response budget: the gain ranges from 0.72 to 3.61 points under SFT+RL and from 1.32 to 2.52 points under RL. The stratified results locate most of this gain in Linear and Loopy programs. The small NLA subset changes by at most two points, corresponding to a single program, and often ties.

D Variability Across Training and Inference Runs

D.1 Protocol and aggregate results

We evaluate three independent training runs of Qwen3-8B, denoted `train-1`, `train-2`, and `train-3`, with three independent inference runs per trained model, denoted `run-1`, `run-2`, and `run-3`. All training and inference settings are held fixed across the nine training–inference combinations on the same 832-program benchmark at response budgets $k \in \{1, 4, 8, 16, 32\}$. Each mean and standard deviation below is computed across the three inference runs of one trained model; it describes inference variability conditional on that training run. The three trained models are reported separately to expose variation across training runs. All rates are percentages, and standard deviations are in percentage points. Results are reported to one decimal place.

Training	@1	@4	@8	@16	@32
<i>Compose@k</i>					
train-1	67.6 ± 0.8	75.7 ± 0.5	77.3 ± 0.6	78.0 ± 0.3	78.1 ± 0.4
train-2	67.6 ± 0.3	75.8 ± 0.3	77.3 ± 0.2	78.3 ± 0.3	78.4 ± 0.3
train-3	68.4 ± 0.2	75.4 ± 0.3	76.8 ± 0.3	77.7 ± 0.2	78.8 ± 0.2
<i>Pass@k</i>					
train-1	36.9 ± 1.0	51.8 ± 0.1	56.1 ± 0.3	59.5 ± 0.1	62.0 ± 0.8
train-2	37.7 ± 0.4	51.7 ± 0.1	56.0 ± 0.1	59.7 ± 0.2	62.8 ± 0.1
train-3	31.7 ± 0.3	43.3 ± 0.4	46.5 ± 0.1	49.8 ± 0.1	53.8 ± 0.2

Table 14: Verification by training run (mean ± SD over three inference runs).

Training	Inference	@1	@4	@8	@16	@32
<i>Compose@k</i>						
train-1	run-1	68.5	75.7	77.5	78.2	78.4
train-1	run-2	66.9	75.2	76.6	77.6	77.6
train-1	run-3	67.3	76.2	77.8	78.1	78.2
train-2	run-1	67.5	76.1	77.5	78.6	78.7
train-2	run-2	67.9	75.8	77.3	78.2	78.4
train-2	run-3	67.4	75.5	77.0	78.0	78.1
train-3	run-1	68.4	75.1	76.4	77.5	78.6
train-3	run-2	68.6	75.6	76.8	77.6	78.7
train-3	run-3	68.3	75.6	77.0	77.9	79.0
<i>Pass@k</i>						
train-1	run-1	36.8	51.9	56.5	59.5	61.5
train-1	run-2	35.9	51.7	56.0	59.5	61.5
train-1	run-3	37.9	51.7	55.9	59.6	62.9
train-2	run-1	38.0	51.8	56.1	60.0	62.7
train-2	run-2	37.9	51.8	56.0	59.6	63.0
train-2	run-3	37.3	51.6	55.9	59.6	62.7
train-3	run-1	32.0	42.9	46.4	49.8	53.6
train-3	run-2	31.6	43.6	46.6	49.9	53.7
train-3	run-3	31.5	43.5	46.5	49.8	54.0

Table 15: Verification for all nine training–inference combinations.

D.2 Results by program class

Tables 16 and 17 report the same within-training inference summaries for Linear (316 programs), NLA (50 programs), and Loopy (466 programs).

Training	@1	@4	@8	@16	@32
<i>Linear</i>					
train-1	72.3 ± 1.0	79.9 ± 0.7	81.1 ± 0.4	81.8 ± 0.5	81.8 ± 0.5
train-2	69.9 ± 0.3	79.2 ± 0.2	80.7 ± 0.0	81.3 ± 0.0	81.3 ± 0.0
train-3	73.7 ± 0.3	79.0 ± 0.7	80.3 ± 0.8	81.4 ± 0.5	81.9 ± 0.5
<i>NLA</i>					
train-1	12.0 ± 2.0	12.7 ± 1.2	13.3 ± 2.3	13.3 ± 2.3	13.3 ± 2.3
train-2	12.0 ± 0.0	14.0 ± 0.0	14.0 ± 0.0	14.0 ± 0.0	14.0 ± 0.0
train-3	14.0 ± 0.0	14.0 ± 0.0	14.0 ± 0.0	14.0 ± 0.0	15.3 ± 1.2
<i>Loopy</i>					
train-1	70.4 ± 1.3	79.7 ± 0.5	81.5 ± 1.1	82.4 ± 0.6	82.5 ± 0.7
train-2	72.0 ± 0.2	80.1 ± 0.5	81.8 ± 0.4	83.1 ± 0.5	83.3 ± 0.5
train-3	70.7 ± 0.1	79.6 ± 0.0	81.1 ± 0.0	82.0 ± 0.0	83.5 ± 0.0

Table 16: Compose@ k by program class (mean ± SD over three inference runs).

Training	@1	@4	@8	@16	@32
<i>Linear</i>					
train-1	39.6 ± 1.5	56.2 ± 0.2	60.1 ± 0.8	63.1 ± 0.5	65.7 ± 0.7
train-2	40.3 ± 0.4	55.7 ± 0.6	59.3 ± 0.5	64.6 ± 0.3	66.9 ± 0.4
train-3	33.1 ± 0.7	45.5 ± 1.0	49.7 ± 0.3	54.2 ± 0.2	57.1 ± 0.5
<i>NLA</i>					
train-1	4.0 ± 0.0	5.3 ± 1.2	6.7 ± 1.2	6.7 ± 1.2	8.0 ± 0.0
train-2	6.0 ± 0.0	6.0 ± 0.0	8.0 ± 0.0	8.0 ± 0.0	8.0 ± 0.0
train-3	2.0 ± 0.0	4.0 ± 0.0	4.0 ± 0.0	4.0 ± 0.0	4.0 ± 0.0
<i>Loopy</i>					
train-1	38.6 ± 1.4	53.7 ± 0.2	58.7 ± 0.9	62.8 ± 0.1	65.2 ± 0.9
train-2	39.3 ± 0.4	53.9 ± 0.2	58.9 ± 0.3	62.0 ± 0.4	66.0 ± 0.2
train-3	33.9 ± 0.0	46.1 ± 0.0	48.9 ± 0.0	51.7 ± 0.0	56.9 ± 0.0

Table 17: Pass@ k by program class (mean ± SD over three inference runs).

Inference variability and differences across training runs. On the full benchmark, the reported within-training standard deviations are at most 0.8 percentage point for compose@ k and 1.0 point for pass@ k . The three training-run means are closely grouped for composition: 67.6–68.4% at compose@1 and 78.1–78.8% at compose@32. Standalone success varies more across training runs, with mean pass@1 ranging from 31.7% to 37.7% and mean pass@32 from 53.8% to 62.8%. Thus, in these runs, similar compositional verification rates coexist with larger differences in standalone success.

E Medium-Reasoning Tool Comparison

Table 18 reports an archived rerun of the main paper’s tool comparison with `reasoning_effort=medium` for every LLM-based row, on the same target-hidden inputs and under the same restored-target judge as the reasoning-disabled comparison in the main paper. These rows use each method’s native token costs and are contextual evidence; they are not mixed into the reasoning-disabled comparison. LORIS and LaM4Inv were evaluated only with reasoning disabled and do not appear in this rerun.

Table 18: Archived target-hidden tool comparison with GPT-5-nano at `reasoning_effort=medium`. Dashes denote unsupported categories. Daikon makes no model call and is listed once for reference. The CRAFT compose rows are prefix compositions of the same archived ten-response pool, measured with the current inference filter chain as in the main paper’s reasoning-disabled comparison.

Method	Linear	NLA	Loopy	Supported Total	Tokens/task	Time/task
AutoSpec	65.82%	8.00%	64.81%	61.78%	25,154.54	54.77 s
SESpec	56.96%	6.00%	49.57%	49.76%	44,807.05	117.31 s
Clause2Inv	19.94%	2.00%	—	17.49%	385.80	29.04 s
Loopy	72.78%	12.00%	70.82%	68.03%	111,973.15	381.01 s
Naive	48.73%	8.00%	47.42%	45.55%	4,493.37	28.87 s
Daikon	23.10%	0.00%	21.67%	20.91%	0	4.89 s
CRAFT (compose@1)	56.96%	54.00%	59.23%	58.05%	6,895.34	60.97 s
CRAFT (compose@4)	76.90%	68.00%	77.90%	76.92%	24,986.02	64.92 s
CRAFT (compose@8)	79.11%	70.00%	82.40%	80.41%	49,106.91	68.98 s
CRAFT (pass@1)	61.08%	14.00%	54.08%	54.33%	6,895.34	53.68 s

Reasoning changes the ranking. Relative to the reasoning-disabled comparison in the main paper, AutoSpec rises from 42.19% to 61.78% and Loopy from 18.75% to 68.03%, reversing their reasoning-disabled ranking; Naive rises from 16.47% to 45.55%. CRAFT benefits as well: compose@1 rises from 33.89% to 58.05% and compose@8 from 55.53% to 80.41%, with NLA compose@1 rising from 22.00% to 54.00%. On Linear, medium compose@1 sits below medium pass@1 (56.96% versus 61.08%); finite-budget filtering need not dominate standalone verification on every stratum, consistent with the filtering-order comparison in the main paper. Under this setting, CRAFT compose@4 verifies 76.92% of programs with 24,986 tokens per task, exceeding every medium-reasoning tool row at lower token cost (Loopy: 68.03% at 111,973 tokens). Reasoning tokens dominate these costs: the archived CRAFT pool averages 6,030 completion tokens per rollout, versus 256 with reasoning disabled. This cost gap motivates the reasoning-disabled main protocol described in the main paper.

Cost accounting follows the main paper’s tool comparison. CRAFT tokens at budget k are the mean prompt per call (865.11) plus k times the mean completion per rollout (6,030.23) billed in the archived pool; time at k is the per-call latency (53.68 s) plus the measured prefix filter-and-judge time (7.29, 11.24, and 15.30 s at $k = 1, 4, 8$). AutoSpec’s token mean uses 832 exact-usage tasks; SESpec’s token mean covers 830 rows with usage records; Clause2Inv’s token mean is estimated over its supported tasks, and its archived medium run covers Linear and NLA (366 tasks), with Loopy unsupported.

F Filtering and Pooling Across Responses

F.1 Matched evaluation and metrics

Table 19 uses a paired evaluation of a Qwen3-8B model trained with SFT and the Full RL reward. All three metrics use the same 832 programs and saved response prefixes. This run is distinct from the selected Full evaluation in the main paper (69.23% $\text{compose}@1$): its $\text{compose}@1$ is 68.51%. The compose and pass rows are also used in Appendix A; the filter rows report the corresponding separately filtered evaluation.

For each response, $\text{pass}@k$ asks whether any of the first k unfiltered responses proves the target. $\text{filter}@k$ applies the same clause admission and Houdini filter to each response separately, then asks whether any one of the first k filtered responses proves the target. $\text{compose}@k$ pools clauses from the first k responses, filters the pooled set once, and checks its conjunction. Thus $\text{filter}@k$ still combines clauses *within* a response; the difference from $\text{compose}@k$ concerns only cross-response pooling and the verifier obligations it creates.

Table 19: Matched verification rates for Qwen3-8B with SFT and Full RL (%). All three $@k$ metrics use the same saved prefixes.

Subset	Metric	@1	@4	@8	@16	@32
All (832)	pass	36.78	51.92	56.49	59.50	61.54
	filter	68.51	75.24	77.52	79.09	80.29
	compose	68.51	75.72	77.52	78.25	78.37
Loopy (466)	pass	37.55	53.86	59.66	62.88	64.81
	filter	71.46	78.97	81.76	83.05	84.55
	compose	71.46	79.61	81.97	82.62	82.83
Linear (316)	pass	40.82	56.33	59.49	62.66	65.19
	filter	73.10	79.43	81.33	83.54	84.49
	compose	73.10	80.06	81.33	82.28	82.28
NLA (50)	pass	4.00	6.00	8.00	8.00	8.00
	filter	12.00	14.00	14.00	14.00	14.00
	compose	12.00	12.00	12.00	12.00	12.00

F.2 What the comparison shows

At one response, $\text{filter}@1$ and $\text{compose}@1$ coincide by construction: both admit and filter the clauses in that response. Their 68.51% success rate is 31.73 percentage points above $\text{pass}@1$. This is the effect of retaining useful clauses from otherwise unsuccessful complete responses, not a gain from combining different responses. After 32 responses, $\text{pass}@32$ reaches 61.54%, $\text{filter}@32$ reaches 80.29%, and $\text{compose}@32$ reaches 78.37%.

The net target-verification difference between $\text{filter}@k$ and $\text{compose}@k$ is small in this trained-model run: compose is 0.48 points higher at $k = 4$, equal at $k = 1, 8$, and 0.84 and 1.92 points lower at $k = 16, 32$. These results do not establish that pooling is logically weaker. Under exact Houdini, filtering the union retains separately inductive clauses and may preserve clauses that require cross-response support. The reversal at $k = 16, 32$ is explained by finite-budget verification: pooling more responses creates larger SMT proof obligations, which are more likely to time out and cause conservative clause deletion. Separately filtering each response keeps those obligations smaller, so $\text{filter}@k$ can exceed $\text{compose}@k$ despite the ideal logical inclusion. This solver effect is also visible in the main paper’s filtering-order experiment, where increasing the pooled verifier budget recovers verification rate.

Pooling can join separately inductive summaries that prove different parts of a target, or make mutually supporting clauses inductive. For target success in this trained-model run, however, most of the benefit over pass is already available by filtering each response separately.

Filtering and composition are not independent sources of benefit in CRAFT: inference always pools admitted clauses and filters them, even at $k = 1$. When a response contains many clauses, filtering has more scope to recover a useful conjunction; with fewer clauses per response, cross-response pooling has more scope to supply missing constraints. A well-trained CRAFT model often puts many clauses in one response, making filtering dominant here. This explains the pass – compose tension: a complete response can fail while its filtered clauses prove the target. $\text{Compose}@1$ is our primary metric because it evaluates this full procedure on one response.

This table measures target verification, not invariant strength. CRAFT still pools and filters all available clauses to seek the strongest inductive summary supported by the candidate pool within its verifier budget.